

Received 26 July 2024, accepted 12 August 2024, date of publication 15 August 2024, date of current version 26 August 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3444042

RESEARCH ARTICLE

Dynamic Filter Application in Graph Convolutional Networks for Enhanced Spectral Feature Analysis and Class Discrimination in Medical Imaging

ARYAN SINGH^{1,2}, PEPIJN VAN DE VEN^{1,2}, CIARÁN EISING^{1,2}, (Senior Member, IEEE), AND PATRICK DENNY^{1,2,3}, (Member, IEEE)

¹Department of Electronic and Computer Engineering, University of Limerick, Limerick, V94 T9PX Ireland

²Data-Driven Computer Engineering Research Centre, University of Limerick, Limerick, V94 T9PX Ireland

³Department of Computer Science and Information Systems (CSIS), University of Limerick, Limerick, V94 T9PX Ireland

Corresponding author: Aryan Singh (aryan.singh@ul.ie)

ABSTRACT Graph Neural Networks (GNNs) offer a promising direction for medical image analysis, particularly due to their ability to capture complex relationships and handle non-Euclidean data structures often encountered in this domain. This study aims to demonstrate the potential of GNNs in medical image classification, showcasing their comparable performance to leading CNNs while using significantly fewer parameters. We introduce a novel GNN model, Graph Convolution Neural Network Enhanced Connectivity (GCNN-EC), incorporating an enhanced connectivity technique that leverages interrelationships between RGB channel features. Evaluated on the MedMNIST dataset, a standardized benchmark for medical image analysis, GCNN-EC performs competitively against pre-trained DNNs, highlighting its potential as an efficient and effective approach for medical image analysis. Importantly, we do not claim that GCNN-EC outperforms all CNNs in every scenario, but rather aim to showcase the potential of GNNs as a valuable tool in the medical imaging toolkit that can be used on its own or in conjunction with CNNs to further advance the state of the art. Additionally, our model outperforms traditional GNNs on standard image datasets like MNIST and CIFAR-10, mitigating the common issue of over-smoothing. These findings underscore the effectiveness of GCNs with dynamic filtering and encourage further research into GNN-based approaches for medical imaging and broader computer vision tasks.

INDEX TERMS GNN, GCN, CNN, over-smoothing.

I. INTRODUCTION

In the field of image analysis, CNNs have emerged as a powerful tool, particularly within the domain of medical imaging. Their success stems partly from their ability to learn spatial features effectively from grid-like image data [1], [2], a common characteristic of many medical images. However, for certain complex medical image modalities and representations that extend beyond simple Euclidean space, CNN architectures may not be optimal. This has paved the way for the exploration of alternative models, such as GNNs, which can handle more intricate data structures.

The associate editor coordinating the review of this manuscript and approving it for publication was Humaira Nisar¹.

Our research focuses on applying GCNs to medical image analysis tasks, leveraging their potential advantages in dealing with non-Euclidean data. This form of structured data frequently employs graphs for representation [3], [4]. The early GCN approaches relied on spectral domain convolutions, defined in the graph's Fourier space [5], recent advancements have led to spatial domain convolution methods, offering more flexibility and computational efficiency. Graph-based filtering techniques, increasingly applied in image processing, involve constructing graphs to define relationships between image pixels. These methods extend beyond the restriction of physical proximity. Graphs are formed using both non-local and semi-local connections, thus enabling links between pixels that consider more than

just spatial closeness. As highlighted in [6] and [7], these techniques excel at identifying and adapting to edges and textures in images, offering effective approaches to image analysis. Critically, these filtering techniques pave the way for the application of graph convolutions within GCNs. Convolution as a specific filtering operation defined on the graph allows GCNs to leverage the rich information captured in the graph structure. By aggregating information from neighboring nodes and their features, GCN convolutions can refine the representation of each pixel, considering its context within the broader image.

While GNNs have shown promise in various domains, their application to image analysis tasks, especially those involving grid-structured data like images, presents unique challenges.

Our study introduces a novel Graph Convolutional Neural Network with Enhanced Connectivity (GCNN-EC). Inspired by graph-based image segmentation techniques like normalized cuts, GCNN-EC aims to improve feature representation within GCNs for enhanced structural understanding of images. GCNN-EC leverages n-hop neighbors for both feature aggregation and edge prediction. Feature aggregation collects information from nodes within a defined neighborhood, while the edge prediction mechanism utilizes both pixel coordinates and existing node features to establish new connections that capture meaningful image structures. This optimized structural awareness mitigates the well-known issue of over-smoothing in the context of graph representation of images. Our method (GCNN-EC) centers on dynamically adapting edge features to significantly improve node interaction. We enhance graph representation by focusing on long-range feature aggregation and new edge assignment. These dynamic edge and node features, integrated into GCN, demonstrate superior performance in graph-level classification tasks.

To evaluate the robustness of our GCNN-EC model, we primarily utilize the MedMNIST dataset [8], a diverse benchmark of 2D medical images is detailed in Table 1. Additionally, we demonstrate its broader applicability with experiments on established datasets like CIFAR-10, MNIST, and MSTAR 10 classes.

The contributions of this work are as follows:

- **Dynamic Edge Feature Adaptation in GCNs:** The proposed GCNN-EC introduces a novel mechanism to dynamically adapt edge features, significantly improving the representation of relationships between image elements. This enhances the understanding of image structures and long-range dependencies.
- **Mitigation of Over-Smoothing in Graph Representations for Enhanced Image Representation:** GCNN-EC mitigates the common issue of over-smoothing in graph representations of images by emphasizing the importance of meaningful connected components. This ensures that information propagates effectively within relevant regions, allowing GNNs to capture distinct features and generate accurate image representations.

- **Robust Performance Across Diverse Image Datasets:** Evaluation on a variety of datasets, including medical images (MedMNIST) and established benchmarks (CIFAR-10, MNIST, MSTAR), demonstrates GCNN-EC's strong performance and generalizability across different image domains. This showcases its potential for real-world computer vision applications. GCNN-EC significantly outperforms well-established GNNs across most datasets in this study.
- **Comparison with popular DNNs:** We have provided comparison with the popular DNN architectures such as DenseNet121, ResNet18, and EfficientNet-B0, along with a simple CNN model. This comparison is to demonstrate the effectiveness of the proposed method with 100 times fewer parameters.

The remainder of this paper is organized as follows: Section II provides an overview of CNNs, which are the most popular models in computer vision and then introduces GNNs, tracing their development from Graph Signal Processing (GSP) and highlighting their relevance to image analysis tasks. Section III delves into the limitations of GCNs, with a particular focus on the issue of over-smoothing. Section IV presents our proposed GCNN-EC model, detailing its novel dynamic filter-based features and the architecture of our graph classification network. Section V outlines the experimental setup, including the datasets used, implementation details, and evaluation metrics. The results of our experiments are presented and analyzed in Section VI. Finally, Section VII concludes the paper with a summary of our key findings and proposes potential avenues for future research.

II. BACKGROUND AND RELATED WORK

CNNs have achieved remarkable success in image analysis, including in the medical domain. Their ability to extract hierarchical features from structured image data has made them the dominant paradigm for numerous vision tasks. To effectively benchmark the performance and advantages of graph-based approaches like our proposed GCNN-EC model, it is essential to establish a performance baseline using leading CNN architectures. This comparison offers a reference point for understanding the potential gains offered by GNN-based methods, particularly when dealing with the complexities of medical imagery. In this section, we briefly examine several state-of-the-art CNN models recognized for their robust performance in medical image classification.

A. CNNs

CNNs, as outlined in [9], are named after the convolution operation, involving the overlay of a kernel onto an image grid, which is slid across the grid to extract local pixel information. This operation involves a dot product between the filter and the corresponding elements of the image grid, the results of which are stored in an output matrix often termed a feature map or convolved feature.

The convolution operation systematically extracts spatial hierarchies or patterns, transitioning from local pixel details to broader abstract concepts as the network depth increases. The features extracted from convolution and pooling layers are flattened into a vector, serving as the input for tasks such as classification, segmentation, or feature localization. CNNs have been a cornerstone in the field of computer vision [10], [11], demonstrating remarkable performance in tasks such as image classification, object detection, and semantic segmentation, including applications in medical imaging. Recent research has explored the integration of GNNs with CNN-based features and the use of standalone GNNs in medical applications, demonstrating their potential to capture complex relationships and enhance performance in tasks such as medical image classification and analysis [12], [13]. This growing body of work underscores the potential of GNNs as a valuable tool in the medical imaging domain. In our study, we have used three state-of-the-art Deep Convolutional Neural Networks (DCNNs) like ResNet (known for residual blocks) [14], DenseNet (maximizing feature reuse) [15], and EfficientNet (offering superior computational efficiency) [16], to establish performance benchmarks.

While CNNs have undoubtedly been a cornerstone in the field of computer vision, demonstrating remarkable success in tasks such as image classification, object detection, and semantic segmentation [10], [11], they are not without their limitations. Research has highlighted that CNNs can be susceptible to failure when presented with inputs that deviate from the training distribution [17], which can hinder their generalizability to new and unseen data [18]. Moreover, the lack of inherent explainability in CNNs makes it challenging to interpret their decision-making process [19], a critical concern in domains like medical imaging where understanding the rationale behind a model's predictions is paramount. Additionally, CNNs can face challenges in adapting to varying color distributions [20], potentially impacting their performance across diverse datasets [21]. These limitations underscore the ongoing need for research to enhance CNN performance and address issues related to robustness, interpretability, and adaptability in complex image analysis tasks. This has led to a growing interest in exploring alternative or complementary approaches. The unique capabilities of GNNs in capturing complex relationships between features and diverse modalities, as demonstrated in recent investigations [22], offer a promising direction for advancing image analysis capabilities. Studies have shown the potential of GNNs when used alongside CNNs [13] and as standalone models [12], highlighting their effectiveness in various computer vision domains.

The forthcoming section will explore GNNs in-depth, focusing on their fundamental components, historical development, and the challenges they face. We will also discuss various strategies developed to address these challenges [23], [24], emphasizing their applications in medical imaging [25].

B. GNNs

GNNs extend the principles behind convolutional operations in traditional CNNs to the realm of non-Euclidean data represented as graphs. A graph, denoted as $G = (N, E)$, consists of a set of nodes (or vertices) N , and a set of edges E representing connections between them. Edges are formulated as $E \subseteq \{(u, v) \mid u, v \in N\}$. For simplicity, we will focus on undirected graphs where edge ordering does not matter: (u, v) is the same as (v, u) .

In image analysis, nodes often represent pixels or regions, while edges might encode spatial relationships, intensity similarities, or other relevant connections. Both nodes and edges can be characterized by features encapsulating their properties. For example, a node feature in a medical image might include pixel intensity or tissue type, while an edge feature could represent the distance between regions or the presence of a boundary. We will refer to a node feature for node n as f_n , and an edge feature between nodes u and v as $f_{(u,v)}$.

A fundamental concept in GNNs is the adjacency matrix, denoted as A . Each element A_{ij} indicates whether a direct connection (edge) exists between nodes i and j . GNNs leverage the adjacency matrix, along with node and edge features, to learn powerful representations of the graph structure, enabling them to solve various image analysis tasks [26].

GNNs have emerged from a rich lineage of research, drawing inspiration from the principles of signal processing, particularly GSP. This perspective empowers us to understand how information propagates and interacts across the intricate connections within a graph structure. Within the broad spectrum of GNNs, we will primarily focus on GCNs as a prominent example. GCNs directly adapt the convolution operation for graph-structured data and have demonstrated remarkable performance in numerous applications. To gain a comprehensive understanding, we will also explore other well-established GNN architectures like Graph Isomorphism Networks (GINs) and Graph Attention Networks (GATs). These additional models will serve as valuable points of comparison in our later discussions, particularly when evaluating their effectiveness for computer vision tasks.

1) GRAPH CONVOLUTIONAL NETWORKS

(GCNs) enhance the capabilities of Convolutional Neural Networks (CNNs) by extending their powerful feature extraction capabilities to data represented in graph structures. This allows for the processing of non-Euclidean data, which is prevalent in various fields such as social network analysis, neuroscience (specifically in brain connectivity graphs), and computer vision applications involving meshes.

Formally, a GCN can be described as a function $f(X, A)$, where:

- **Node Feature Matrix X :** This is an $N \times F$ matrix, where N represents the number of nodes, and F is the number of features per node. Examples of node features include:

- 1) **RGB Values & Coordinates:** These represent the pixel intensities and their respective locations within an image.
- 2) **Haralick Features:** These are statistical texture measures that are derived from specific regions of an image.
- 3) **CNN-extracted Features:** These high-level representations are obtained from pre-trained CNN models.
- **Adjacency Matrix A :** Previously introduced, the $N \times N$ matrix, which denotes the connections between nodes. It is often normalized to ensure the proper scaling of features, as discussed in various sources, such as [27].

GCNs update node representations through layer-wise propagation, taking into account the information from neighboring nodes. A simplified representation of this propagation rule is illustrated in (1):

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right). \quad (1)$$

where:

- $\tilde{A} = A + I$ is the adjacency matrix with added self-connections.
- $\tilde{D}$ is the degree matrix corresponding to $\tilde{A}$.
- $W^{(l)}$ is the layer-specific trainable weight matrix with dimensions $F^{(l)} \times F^{(l+1)}$.
- $H^{(l)}$ represents the node features at layer l .
- $\sigma(\cdot)$ is a non-linear activation function.

GCNs are inspired by GSP, a discipline that focuses on analyzing signals defined on the nodes or edges of graphs. A fundamental component in GSP is the graph Laplacian, with its unnormalized form defined as $L = D - A$, where D represents the degree matrix.

The Laplacian matrix is crucial for several reasons:

- **Structural Insights:** It encodes vital information about the graph's structure, with its eigenvalues and eigenvectors revealing essential characteristics such as community presence, bottlenecks, and other significant structural patterns within the graph.
- **Normalization's Role:** Techniques like the symmetric normalized Laplacian ($L_{sym} = D^{-1/2} A D^{-1/2}$) are essential for achieving a stable and comparable spectral representation. This normalization is particularly important to prevent high-degree nodes from dominating the spectrum, which could hinder the analysis of other nodes.

Further, the graph Fourier transform, derived from the eigendecomposition of the Laplacian ($L = U \Lambda U^T$), facilitates the analysis of graph signals in a spectral domain, analogous to traditional Fourier analysis. In this domain, the eigenvectors of the Laplacian form a basis for a space akin to Fourier analysis. This approach allows for the dissection and understanding of a graph signal's frequency components, enabling tasks like signal filtering, pattern identification, and comprehending signal smoothness in relation to the graph's structure [28].

Whilst graph signal processing is a powerful technique for filtering, compression, and representation learning [28], it is associated with a high computational complexity of $\mathcal{O}(n^3)$. GCNs provide an innovative solution to the primary limitation of GSP and it achieves this by efficiently approximating spectral convolutions on graphs, utilizing the mathematical properties of Chebyshev polynomials.

This methodological shift has profound implications for how we process graph-structured data. By avoiding the expensive eigendecomposition of the Laplacian matrix, GCNs facilitate a more scalable and efficient means of performing deep learning on graphs. The underlying principles and mechanics of this approximation, particularly the use of Chebyshev polynomials in redefining spectral convolutions, will be examined in greater detail in the following sections. This discussion will delve into the theoretical underpinnings of GCNs and explore how these networks manage to retain the depth and effectiveness of traditional GSP approaches while dramatically reducing their computational burden.

GCNs achieve this by adapting the approach used in ChebNet [29] by employing only the first two terms of the Chebyshev polynomial expansion, effectively setting $K = 1$. Chebyshev interpolation, as explained in [29], is a method to approximate a continuous filter function $h(\hat{\lambda})$ known at a finite number of points $\hat{\lambda}_k$ by a polynomial P_K of degree K such that $h(\hat{\lambda}_k) = P_K(\hat{\lambda}_k)$. This interpolation approach uses Chebyshev nodes rather than uniformly distributed points, leading to faster convergence and avoiding issues like the Runge Phenomenon. The Chebyshev nodes are the zeros of the $(K + 1)$ -th Chebyshev polynomial (The zeros of a Chebyshev polynomial of a certain degree are the points at which the polynomial equals zero), providing an efficient and effective means for polynomial interpolation in spectral graph analysis.

The convolution operation in GCNs is thus represented as:

$$g_\theta * x \approx \sum_{k=0}^K \theta_k T_k(\tilde{\Lambda}) x \quad (2)$$

where g_θ is the spectral filter function with learnable parameters θ . $*$ denotes the convolution operation. x is the graph signal (e.g., node features). $T_k(\tilde{\Lambda})$ is the Chebyshev polynomial of order k evaluated at the scaled eigenvalue matrix $\tilde{\Lambda}$ of the graph Laplacian.

However, directly computing (2) has high computational complexity due to eigendecomposition. GCNs address this by approximating spectral convolutions. Using a truncated Chebyshev expansion (setting $K = 1$) and sharing parameters, we arrive at:

$$g_\theta * x \approx \theta_0 T_0(\tilde{\Lambda}) x + \theta_1 T_1(\tilde{\Lambda}) x \quad (3)$$

Simplifying further by allowing $\theta = \theta_0 = \theta_1$ (which is a design choice in GCNs [27] aimed at reducing the number of learnable parameters and improving computational efficiency) yields:

$$g_\theta * x \approx \theta (I + D^{-\frac{1}{2}} A D^{-\frac{1}{2}}) x \quad (4)$$

This demonstrates how GCN layers efficiently aggregate information from a node's neighborhood. This localized operation significantly reduces computational costs compared to full spectral convolutions.

In this section, we introduced GCNs, tracing their origins in GSP and highlighting the computational advancements that have enabled their successful application to large-scale real-world graphs. This has led to their use in tasks like object localization [30], visual question answering [31], video analysis [31], and image classification [13]. While demonstrating remarkable results, GCNs face limitations similar to those in DNNs, such as vanishing and exploding gradients. These limitations can be addressed using techniques like residual connections, which facilitate the learning of long-term dependencies in deeper architectures [32]. A detailed exploration of GCN limitations is presented in Section III.

III. LIMITATIONS OF GCN

While GCNs have made significant strides in encoding graph structures, they do face limitations that researchers continue to address. Some key categories of limitations include:

- **Over-smoothing:** As GCN layers aggregate information across neighborhoods, node representations can become overly similar, especially in deeper architectures. This hinders the ability to distinguish subtle structural differences [33].
- **Limits of the Weisfeiler-Lehmann Test:** GCNs' discriminative power often aligns with the Weisfeiler-Lehmann (WL) graph isomorphism test [34]. This suggests limitations in distinguishing graph structures that the WL test can differentiate.
- **Long-range Dependencies:** Capturing complex, long-range dependencies within the graph structure can be challenging for standard GCN variants [35]. This has led to the development of attention-based mechanisms like Graph Attention Networks (GATs) [36].
- **Expressive Power and Graph Isomorphism:** While achieving significant progress, even advanced GCNs like Graph Isomorphism Networks (GINs) [37] face theoretical limits in their ability to fully capture all nuances that distinguish non-isomorphic graphs.

The predominant limitation in current GCNN models lies in their constrained ability to represent topology [38]. This limitation is further compounded by the issue of over-smoothing [39], a consequence of iterative feature aggregation from connected nodes, leading to increasingly indistinguishable node embeddings. Such over-smoothing challenges the capability of GCNNs to capture complex structural patterns, particularly those beyond the scope of the 1-WL test (where '1' denotes the consideration of 1-hop neighbors) [40]. A detailed discussion on over-smoothing and its implications on the effectiveness of GCNNs in encoding topological nuances will be further elaborated in this section.

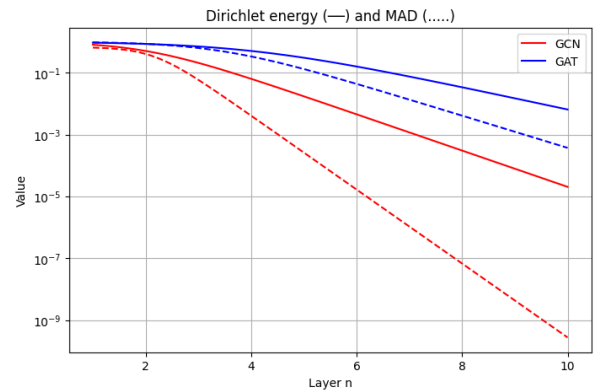


FIGURE 1. Comparison of Dirichlet Energy and MAD across layers for GCN, GAT on synthetic graph data generated using SBM, illustrating the over-smoothing, faster damping of Dirichlet energy means more over-smoothing.

This exploration is critical for enhancing the representation learned by GCNNs.

A. THE CHALLENGE OF OVER-SMOOTHING IN GCNS

GCNs face the inherent challenge of over-smoothing, especially when many convolutional layers are employed. Over-smoothing in GCNs is primarily due to the nature of graph convolution, which essentially acts as a type of Laplacian smoothing [41]. We can define Over-smoothing as layer-wise exponential convergence of the node-similarity measure μ to zero. As more convolutional layers are added, the distinctive features of vertices from different clusters may blend, leading to indistinguishable node representations [33]. Node similarity measures μ can be estimated in two ways [33]:

- Using Dirichlet energy which is obtained by summing the squared differences of features between adjacent nodes.
- Using Mean Average Distance (MAD) which measures feature similarity by averaging the cosine dissimilarity (one minus the normalized dot product) of features between each node and its neighbors.

In graph-level classification, the effect of smoothing is most pronounced. This is because the final graph-level embeddings are derived by aggregating, or pooling, the features of all nodes. Such pooling inherently smooths out the distinctive features of individual nodes across the graph.

B. EMPIRICAL OBSERVATION WITH BREASTMNIST GRAPH DATASET AND SYNTHETIC DATASET

To investigate the over-smoothing phenomenon, we created a synthetic dataset with two distinct blocks of five nodes each. Nodes within a block are connected, but there are no connections between blocks. Each node has five unique features, ensuring clear differentiation between blocks. Our findings confirm that GCNs' message-passing and aggregation mechanisms inherently lead to over-smoothing of node features. The rate of this feature mixing is influenced

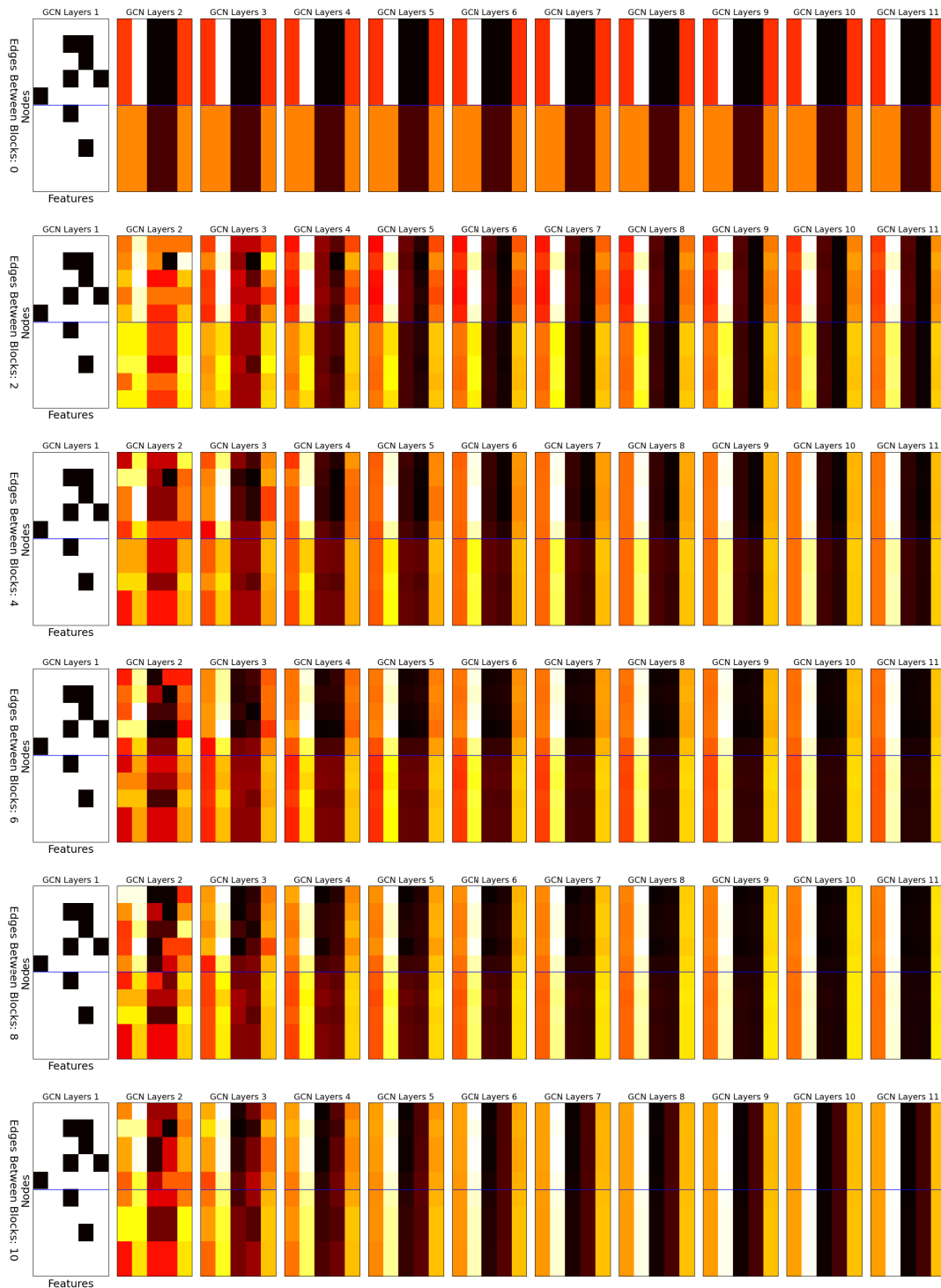


FIGURE 2. Heatmap visualization of GCN node features over multiple layers. The y-axis represents nodes (separated by a blue line between blocks), and the x-axis shows the number of GCN layers. Note the lack of connection between blocks, and the progressive blurring of features as layer count and edges between blocks increase, demonstrating over-smoothing.

by both connectivity and the features themselves as shown in Figure 2. Figure 1 demonstrates this effect using two established over-smoothing metrics – Dirichlet Energy and MAD – on a synthetic graph. Over-smoothing leads to a faster decrease in Dirichlet Energy, indicating a loss of feature distinctiveness.

This adversely affects the downstream task. To illustrate the negative impact of over-smoothing on graph classification, we turn to a real-world example. We see this phenomenon in action with the BreastMNIST dataset. This dataset consists of 780 graphs representing images, where each pixel serves as a vertex connected to its four

neighbors. This creates graphs with 784 vertices and 6,724 edges, divided into two distinct classes. As GCN layers are applied, graph-level features rapidly converge, rendering them indistinguishable. This convergence, driven by the direct correlation between node features and pixel values, hampers graph classification performance (see Figure 3).

C. THEORETICAL INSIGHTS INTO FEATURE CONVERGENCE

Oversmoothing in GCNs has both empirical and theoretical underpinnings. Images transformed into graphs, like the BreastMNIST dataset, often lack structurally significant edges compared to other types of graph data. This leads to the formation of connected components with relatively weak distinctions. Repeated applications of GCNs propagate features within these components, ultimately leading to over-smoothing and hindering classification tasks.

Theoretical analysis reveals that with repeated application of Laplacian smoothing, the features of vertices within each connected component of a graph tend to converge to the same values [41]. For symmetric Laplacian smoothing, this convergence aligns with the square root of the vertex degree. Mathematically, this is expressed as follows:

Suppose that a graph G has k connected components $\{C_i\}_{i=1}^k$, and the indication vector for the i -th component is denoted by $1^{(i)} \in \mathbb{R}^n$, which serves to identify the membership of vertices in the component. This vector indicates whether a vertex is in the component C_i , i.e.,

$$1_j^{(i)} = \begin{cases} 1, & \text{if } v_j \in C_i \\ 0, & \text{if } v_j \notin C_i \end{cases}$$

Mathematically, applying a GCN layer can be represented by multiplying the feature matrix X by a specific transformation matrix. This transformation depends on the Laplacian matrix L of the graph, which encodes the connectivity information, and a hyperparameter α that controls the influence of the Laplacian. Normalization of the Laplacian matrix is often employed to stabilize the numerical computations and control the scale of the feature transformations. The two most common types of normalization used are the random walk Laplacian L_{rw} and the symmetric normalized Laplacian L_{sym} .

After repeated applications of these GCN layers (denoted by m iterations), a fascinating property arises: the node features start converging towards a specific combination of the eigenvectors of the Laplacian matrix being used. We can express this mathematically using the following equations:

$$\begin{aligned} \lim_{m \rightarrow +\infty} (I - \alpha L_{rw})^m X &= [1^{(1)}, 1^{(2)}, \dots, 1^{(k)}] \theta_1, \\ \lim_{m \rightarrow +\infty} (I - \alpha L_{sym})^m X &= D^{-\frac{1}{2}} [1^{(1)}, 1^{(2)}, \dots, 1^{(k)}] \theta_2, \end{aligned}$$

Here, I is the identity matrix, and D is a diagonal matrix containing the degrees of each node in the graph. The key elements are:

- **Eigenvectors:** These are special directions associated with the Laplacian matrix. Their properties influence

how information propagates through the graph during GCN application.

- **Indicator vectors:** $1^{(i)} \in \mathbb{R}^n$ represents a vector where all elements are zero except for the entries corresponding to nodes in the i^{th} connected component of the graph. These indicate the membership of nodes within connected components.
- **Weight vectors:** θ_1 and θ_2 are vectors in $\mathbb{R}^k$ (where k is the number of connected components) that determine the specific contribution of each indicator vector to the final combination.

While convergence seems like a natural consequence of information propagation, it becomes problematic when the number of GCN layers (iterations, m) becomes very large [39]. In this scenario, the features of nodes within the same connected component become increasingly similar and ultimately converge to the same linear combination of the Laplacian's eigenvectors. This phenomenon is called over-smoothing. Consider a graph representation of an image where nodes correspond to pixels or superpixels, and edges denote neighboring relationships. Over-smoothing in this context leads to a blurring of features within distinct image regions or objects. In the following section, we will explore the implications of these limitations for GCN design.

D. IMPLICATIONS FOR GCN DESIGN

The phenomenon of over-smoothing highlights limitations in conventional GCN designs. This underscores the importance of rethinking how graphs are represented and utilized in GNNs. To mitigate over-smoothing, strategies such as skip connections, have been employed:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) + H^{(l)} \quad (5)$$

This allows the network to retain original feature information at each layer, thereby preserving the uniqueness of node embeddings and combating the over-smoothing effect.

Building on this, we explore additional avenues for enhancing GCN architectures:

- **Beyond Local Neighborhoods:** While traditional message-passing schemes often aggregate features from immediate neighbors, we will explore techniques to incorporate information from extended neighborhoods. This allows for the extraction of richer features that capture both local and long-range dependencies within the graph.
- **Structural Refinement:** We will investigate methods to use the learned features and topological insights to modify or create new edges within the graph. This dynamic graph refinement enables the GCN to better represent complex structural and spatial relationships within the image data.

While skip connections help preserve feature distinctiveness, they have limitations when dealing with node attributes like RGB values, spatial coordinates, or Haralick features. These attributes often lack the richness needed to

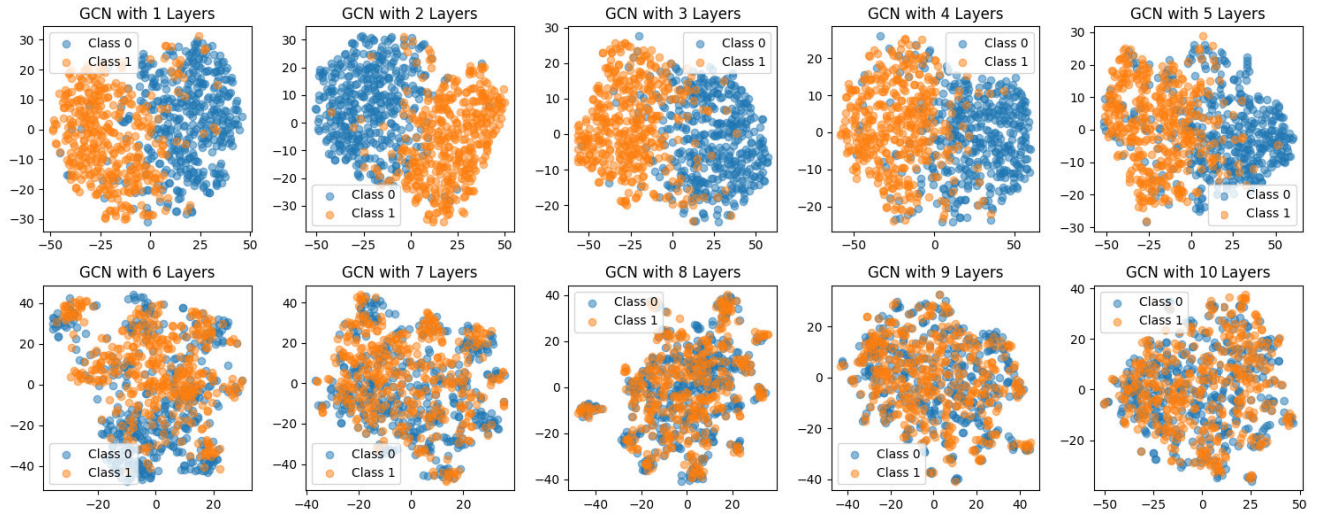


FIGURE 3. GCN graph level embeddings of BreastMNIST binary classes. We can observe that increasing the number of layers leads to feature mixing. This mixing results in graph-level embeddings that become less distinguishable between the two classes.

fully prevent over-smoothing. This underscores the need for a new approach that can better capture and utilize local image patterns. To address this challenge, we introduce an innovative method that dynamically integrates edge features created by applying filters to adjacent nodes. This technique strengthens the model's ability to learn complex patterns around each node, enhancing both connectivity and feature distinction.

IV. METHOD

This section provides a detailed overview of our innovative approach designed to address the challenges faced by conventional GCNs. Central to our method is the dynamic adaptation of edge features, which significantly enhances the interaction between nodes.

Our proposed method harnesses the best of the node features and graph topology by advancing graph connectivity and node interaction. This is achieved by focusing on two ways to enhance graph representation:

- 1) **Long-Range Feature Aggregation:** This function facilitates the gathering of features from distant nodes, beyond the immediate neighbors typically considered in traditional GCN architectures. By aggregating long-range features, the dynamic filter layer enables a broader view of the graph's topology, thereby enriching the node feature set with diverse and contextually relevant information.
- 2) **New Edge Assignment:** Utilizing the aggregated long-range features, this layer dynamically assigns new edges within the graph. This process is geared towards enhancing the shape and structure of the connected components.

We have used a *Dynamic Filter Layer* which is responsible for feature aggregation and edge prediction.

Following this, we have *Dynamic Filter Based Features*, where these dynamically computed edge features are inte-

grated to refine node representations. This integration not only improves the individual node characteristics but also the overall structure of the graph.

Subsequently, these elements are incorporated into the *Graph Classification Network*. This integration demonstrates the combined effectiveness of dynamic edge and node features in graph-level classification tasks.

Our method's pipeline is depicted in Figure 4, which illustrates the sequential processing and integration of these innovative components within our enhanced GCN framework.

A. DYNAMIC FILTER LAYER

Our Dynamic Filter Layer draws inspiration from graph partitioning and spectral clustering, particularly the n-cut method. This layer learns filters that capture multi-scale interactions and predict optimal connectivity, leading to refined graph structures.

We begin with the graph Laplacian, denoted as L . To capture interactions between a node and its n-hop neighbors, we apply a polynomial of the Laplacian with learnable coefficients:

$$p(L) = \sum_{k=0}^K \alpha_k L^k \quad (6)$$

This polynomial is applied to the node feature vectors $\mathbf{x}$:

$$\mathbf{X}_{ij}^e = \text{Filter} \left(p(L)\mathbf{x}_i, \frac{1}{|\mathcal{N}_i^{(n)}|} \sum_{u \in \mathcal{N}_i^{(n)}} p(L)\mathbf{x}_u \right) \quad (7)$$

In the above equation, Filter denotes the operation of applying $p(L)$ to $\mathbf{x}_i$, i.e., the node feature, along with the n-hop neighbors of $\mathbf{x}_i$ denoted by $\mathcal{N}_i^{(n)}$.

(6) and (7) start with the initial graph and then learn new coefficients for Laplacian as in Algorithm 1.

The edge features generated by Filter, denoted as $\mathbf{X}_{ij}^e$, encapsulate the connectivity potential between nodes i

TABLE 1. Summary of MedMNIST-2D datasets including OR: Ordinal Regression. MC: Multi-Class. ML: Multi-Label. BC: Binary-Class.

Name	Data Modality	Task (Classes / Labels)	Samples	Training / Validation / Test
ChestMNIST	Chest X-Ray	ML (14), BC (2)	112,120	78,468 / 11,219 / 22,433
PathMNIST	Colon Pathology	MC (9)	107,180	89,996 / 10,004 / 7,180
OCTMNIST	Retinal OCT	MC (4)	109,309	97,477 / 10,832 / 1,000
DermaMNIST	Dermatoscope	MC (7)	10,015	7,007 / 1,003 / 2,005
RetinaMNIST	Fundus Camera	OR (5)	1,600	1,080 / 120 / 400
PneumoniaMNIST	Chest X-Ray	BC (2)	5,856	4,708 / 524 / 624
BreastMNIST	Breast Ultrasound	BC (2)	780	546 / 78 / 156
TissueMNIST	Kidney Cortex Microscope	MC (8)	236,386	165,466 / 23,640 / 47,280
BloodMNIST	Blood Cell Microscope	MC (8)	17,092	11,959 / 1,712 / 3,421
OrganAMNIST	Abdominal CT	MC (11)	58,850	34,581 / 6,491 / 17,778
OrganCMNIST	Abdominal CT	MC (11)	23,660	13,000 / 2,392 / 8,268
OrganSMNIST	Abdominal CT	MC (11)	25,221	13,940 / 2,452 / 8,829

and j , based on their similarity and affinity. Incorporating features from n -hop neighbors enriches the generation of optimally connected components, laying the groundwork for the graph's initial structure.

Algorithm 1 Learned Laplacian Layer

Require: in_ch, out_ch, deg (default=2)
 $coeff \leftarrow \mathbb{R}^{deg+1}$
 $linear \leftarrow \text{Linear}(in_ch, out_ch)$
Function Forward($x, edge$):
 $edge, norm \leftarrow \text{GetNormalizedLaplacian}(edge)$
 $tx \leftarrow x$
 $out \leftarrow coeff[0] \times tx$
for $i = 1$ **to** deg **do**
 $tx \leftarrow \text{Matmul}(edge, tx, norm)$
 $out \leftarrow out + coeff[i] \times tx$
end for
return $\text{Linear}(out)$

We extend the utility of these features by integrating a Multi-Layer Perceptron (MLP) to project them into a 16-dimensional space for each edge:

$$\mathbf{X}\mathbf{e}'_{ij} = \text{MLP}(\mathbf{X}\mathbf{e}_{ij}) \quad (8)$$

Subsequently, as outlined in Algorithm 2, the transformed edge feature vector $\mathbf{X}\mathbf{e}'_{ij}$ is processed through an activation function, as demonstrated in Algorithm 3, to predict the presence of edges, thus optimizing the graph structure:

$$\mathbf{e}''_{ij} = \text{Activation}(\text{Tanh})(\mathbf{X}\mathbf{e}'_{ij}) \quad (9)$$

Here, $\mathbf{e}''_{ij}$ denotes the output representing the existence of an edge between nodes i and j , essential for refining the graph's structure.

Through the adoption of dynamic filtering concepts from image processing to GNNs, our approach enables filters to dynamically interact with node and edge features. This dynamic interaction facilitates position-specific transformations and adaptability, closely modeling significant node

and edge relationships and aligning with the foundational principles of the n -graph cut method and spectral clustering in a machine learning framework.

Algorithm 2 Dynamic Edge Feature Layer

Require: $in_channels, out_channels$
 $\text{MLP} \leftarrow \text{Sequential}(\text{Linear}(2 \times in_channels, out_channels), \text{ReLU}, \text{Linear}(out_channels, in_channels))$
Function Forward($x, edge_index$):
 $row, col \leftarrow \text{Decompose } edge_index$
 $edge_features \leftarrow \text{Concat}(x[row], x[col])$
 $edge_features \leftarrow \text{MLP}(edge_features)$
 $agg_edge_features \leftarrow \text{ScatterAdd}(edge_features, row)$
return $agg_edge_features$

Algorithm 3 Edge Prediction Layer

Require: in_ch
 $\text{MLP} \leftarrow \text{Sequential}(\text{Linear}(in_ch, 1), \text{Tanh})$
Function Forward($x, edge_idx$):
 $r, c \leftarrow \text{Decompose}(edge_idx)$
 $pred \leftarrow \text{MLP}(x[r] - x[c])$
 $filtered_idx \leftarrow \text{Filter based on } (pred > 0)$
return $filtered_idx, pred$

B. DYNAMIC FILTER BASED FEATURES

The dynamic filter layer enhances the graph by refining edge connections and integrating edge features. These edge features encapsulate long-range regional representations, which are then merged with node features. This fusion serves a dual purpose: while node features capture local characteristics, edge features introduce global or regional context, enriching the detection of distinct communities. Nodes are subsequently updated with this enriched feature set, forming a new graph representation for graph-level classification in our study.

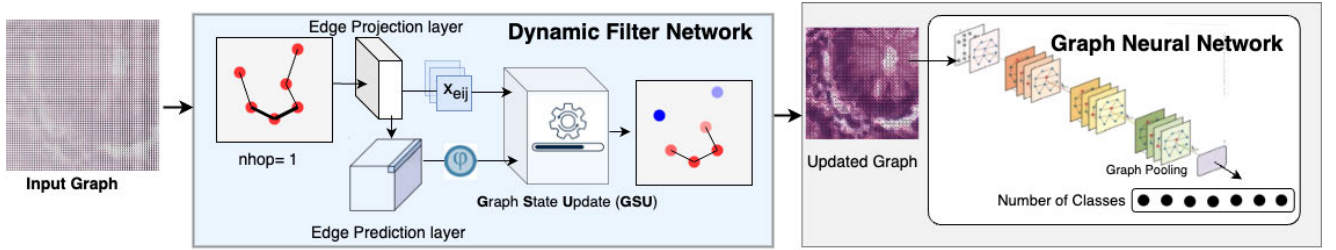


FIGURE 4. Depiction of the *GraphClassificationNetwork* within our model architecture, highlighting the flow from dynamic layers through GCN layers to the global mean pooling layer, and ultimately to the classifier for final predictions.

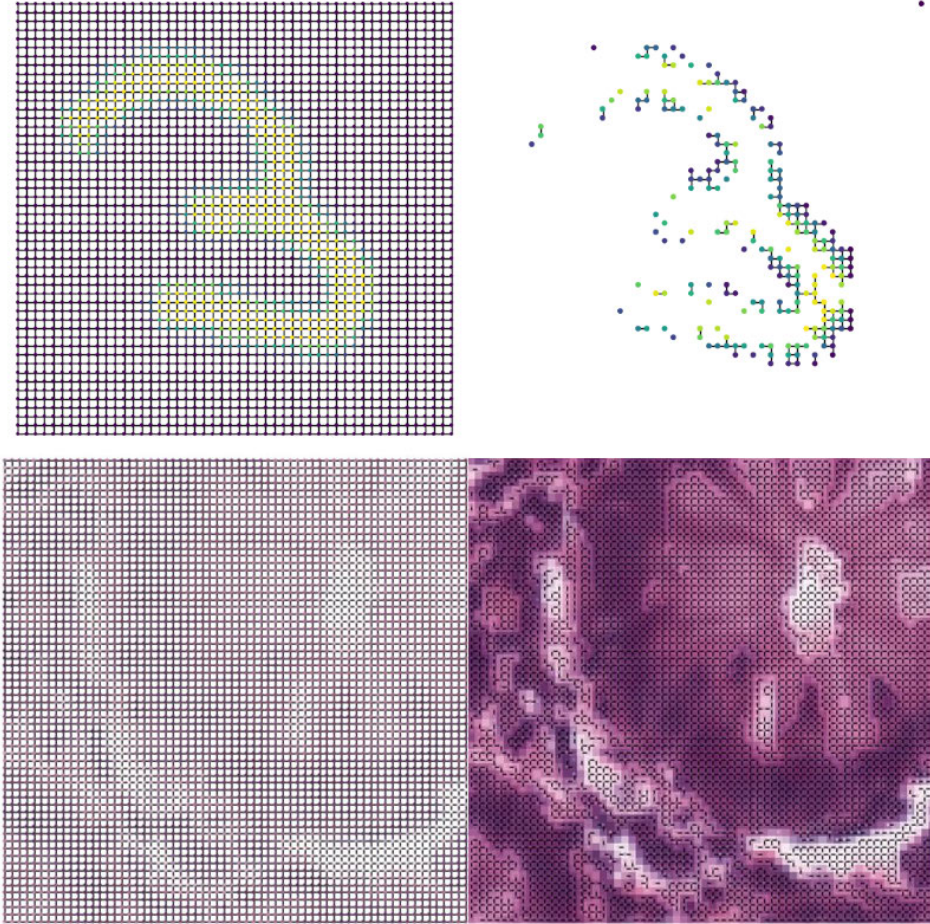


FIGURE 5. Result of Dynamic filter applied to pixels of MNIST (1 channel) top and MedMNIST (3 channel) bottom. The final graph representation, which holds the optimal graph structure, is further passed to GCN layers.

1) UPDATE MECHANISM

Node feature vectors are augmented by concatenating them with the features of their eight adjacent edges. This update is mathematically represented as:

$$\mathbf{X}'_i = [\mathbf{X}_i; \mathbf{X}_{e_{i1}}; \dots; \mathbf{X}_{e_{i4}}] \quad (10)$$

C. GRAPH CLASSIFICATION NETWORK

The *GraphClassificationNetwork* (Figure 4) in our model architecture plays a pivotal role in graph-level classification. This network is an extension of the baseline model [27], which consists of bare-bone GCN layers. Structurally, it com-

bines the Dynamic Filter Layer and Dynamic Filter Features aggregation to enhance the initial graph representation and subsequent GCN layers are responsible for performing deep graph convolution operations. To aggregate node features into a unified graph-level representation, a global mean pooling layer is employed. Finally, the network features a classifier, tasked with generating the final predictions based on these graph-level embeddings.

V. EXPERIMENTAL SETUP

To comprehensively evaluate the effectiveness of our dynamic filtering technique in GCNs for image classifica-

TABLE 2. Comparison of various methods on MedMNIST datasets. The best-performing model for each of the MedMNIST datasets is highlighted (ACC = accuracy, AUC = area under the ROC curve).

Methods	PathMNIST		ChestMNIST		DermaMNIST		OCTMNIST		PneumoniaMNIST		RetinaMNIST	
	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC
DenseNet121	0.983	0.907	0.768	0.947	0.917	0.735	0.943	0.743	0.944	0.854	0.717	0.524
EfficientNetB0	0.990	0.911	0.769	0.947	0.913	0.735	0.952	0.762	0.948	0.854	0.726	0.528
ResNet-18	0.959	0.834	0.742	0.937	0.915	0.749	0.955	0.763	0.947	0.878	0.719	0.503
CNN (2 conv)	0.953	0.739	0.540	0.947	0.816	0.669	0.895	0.566	0.900	0.808	0.665	0.435
GCN	0.641	0.450	0.811	0.630	0.778	0.425	0.488	0.256	0.460	0.625	0.460	0.267
GIN	0.870	0.703	0.811	0.781	0.793	0.661	0.744	0.694	0.451	0.625	0.661	0.437
GAT	0.558	0.410	0.838	0.761	0.661	0.477	0.610	0.400	0.460	0.625	0.511	0.305
GCNN-EC	0.944	0.791	0.937	0.874	0.892	0.755	0.903	0.761	0.954	0.877	0.712	0.430

Methods	BreastMNIST		BloodMNIST		TissueMNIST		OrganAMNIST		OrganCMNIST		OrganSMNIST	
	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC
DenseNet121	0.901	0.863	0.998	0.958	0.930	0.676	0.997	0.935	0.992	0.900	0.972	0.782
EfficientNetB0	0.857	0.812	0.997	0.956	0.931	0.680	0.997	0.935	0.992	0.905	0.972	0.770
ResNet-18	0.871	0.831	0.998	0.961	0.941	0.703	0.994	0.905	0.990	0.879	0.974	0.813
CNN (2 conv)	0.633	0.731	0.954	0.750	0.866	0.559	0.821	0.780	0.947	0.818	0.771	0.553
GCN	0.688	0.419	0.373	0.119	0.377	0.230	0.701	0.435	0.635	0.485	0.741	0.435
GIN	0.713	0.670	0.738	0.504	0.716	0.677	0.594	0.605	0.667	0.451	0.249	0.326
GAT	0.769	0.668	0.511	0.384	0.738	0.529	0.601	0.472	0.260	0.315	0.301	0.228
GCNN-EC	0.865	0.795	0.909	0.869	0.771	0.599	0.937	0.834	0.929	0.751	0.904	0.674

tion, we conducted experiments on four carefully selected benchmark datasets: MNIST, CIFAR-10, MedMNIST and MSTAR-10. Here's a breakdown of our choices:

- **MNIST:** This widely used dataset provides a well-established baseline for evaluating image classification algorithms. While considered a simpler dataset, MNIST allows us to clearly illustrate the core functionalities of our approach and serves as a foundational test case.
- **CIFAR-10:** Building upon MNIST, CIFAR-10 offers a more challenging dataset with higher image resolution and increased class complexity (10 object classes). This selection allows us to assess how our method scales to more intricate image classification tasks.
- **MedMNIST:** MedMNIST includes different types of medical images like X-rays and MRIs, similar to real-world medical situations. It helps test how well our method works on various complex tasks. The details of the 12 2D image datasets are shown in Table 1. MedMNIST is a powerful tool that has been actively used as a benchmark dataset in numerous studies [42], [43], [44], [45], [46], [47], [48], [49], proving to be highly valuable for evaluating model performance and advancing research in medical image analysis. Its utility stems from its diverse collection of standardized medical image datasets, ranging from small to large scales and covering a variety of modalities. The availability of both 28×28 and 224×224 image sizes further enhances its flexibility, allowing for fast benchmarking and comprehensive evaluation of different models.
- **MSTAR-10:** This dataset focuses on the classification of military vehicles in Synthetic Aperture Radar (SAR) imagery. The unique challenges of SAR data, such

TABLE 3. Graph classification results (Accuracy).

METHOD	MNIST	CIFAR-10	MSTAR10
GCN-4	0.901	0.541	0.568
GIN-4	0.964	0.552	0.731
GAT-4	0.955	0.642	0.580
GCNN-EC	0.971	0.661	0.951

TABLE 4. Parameter counts of various models.

METHOD	Feature size	Layers	Parameter (for RGB dataset)
DenseNet121	3*224*224	—	7,978,856
ResNet18	3*224*224	—	11,689,512
EfficientNetB0	3*224*224	—	4,014,658
CNN	3*32*32	conv(2)	155,097
GCN	5	GCNConv(4)	44,194
GIN	5	GINConv(4)	64,198
GAT	5	GATConv(4)	671,586
GCNN-EC	5	GCNN-EC(4)	95,210

as speckle noise and viewpoint variations, provide a distinct test case for our method. Its inclusion expands the scope of our evaluation to a different imaging modality and a domain-specific application.

By using this diverse array of datasets, we aimed to compare the effectiveness of our GCNN-EC model across different domains, varying levels of difficulty, and color schemes. This strategic choice enhances our evaluation, showcasing the model's adaptability and effectiveness in a broad spectrum of image classification challenges, including tasks within the medical field.

A. IMPLEMENTATION DETAILS

In our research, we established a basic model using a simple GCN structure, which includes two layers of GCN convolution (GCNConv). We implemented this model in the PyTorch framework, specifically leveraging the PyTorch

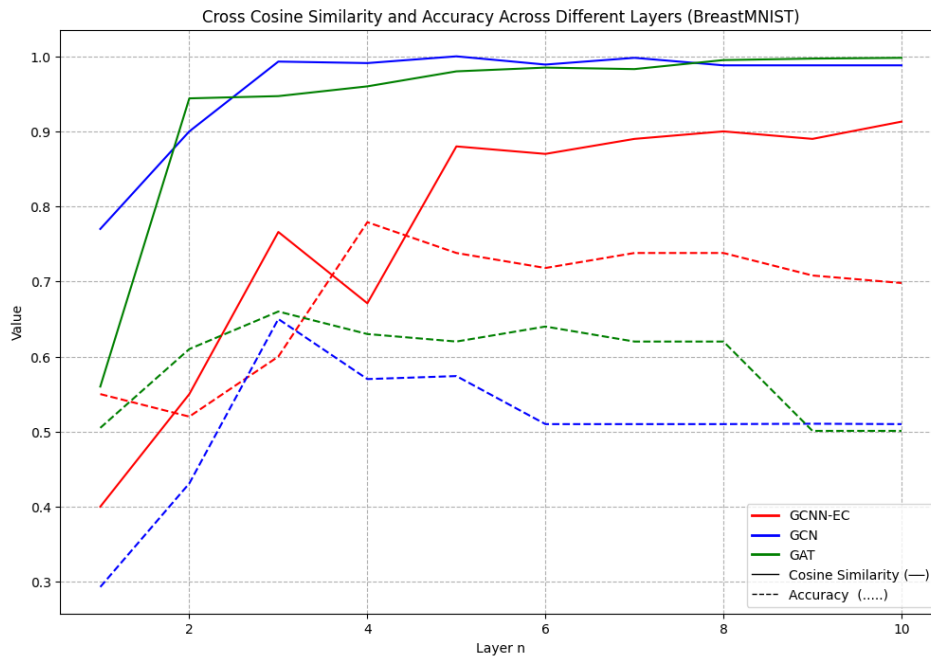


FIGURE 6. Feature Evolution in Layered GNNs for BreastMNIST Graphs: The plot illustrates the cosine similarity across 10 layers in GCN models, contrasting the quick convergence in standard GCN and GAT with the gradual increase observed in GCN-EC. This suggests the potential of GCN-EC to maintain diverse feature learning across layers, avoiding over-smoothing.

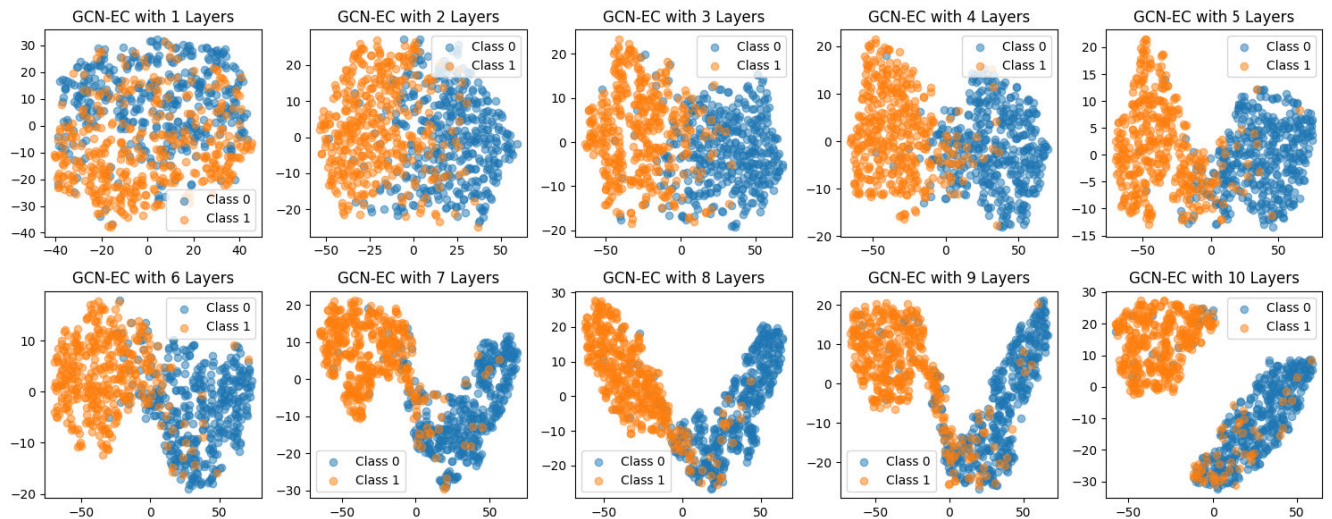


FIGURE 7. GCNN-EC graph level embeddings of BreastMNIST binary classes visualized. The overmixing of features is mitigated, resulting in graph-level embeddings that are more distinguishable between the two classes.

Geometric (PyG) library, recognized for its effectiveness in geometric deep learning applications. We established a GCN model using a four-layer GCN convolution structure (GCNConv). As with our Dynamic Filter + GCN model, this model is implemented within the PyTorch framework, leveraging the PyTorch Geometric (PyG) library. Both models feature four layers, allowing us to directly isolate and evaluate the impact of the Dynamic Filter layer. To construct graphs from images, we represent each pixel as a node. Nodes are connected using a 4-neighbor connectivity pattern. Each node's feature vector incorporates the pixel's RGB values

along with its spatial coordinates (x, y) within the 2D image. These graph representations were formatted for compatibility with the PyG dataloader, facilitating both training and testing phases.

B. EXPERIMENT

In our experiment, the model was trained using a batch size of 64 on an NVIDIA T4 with 12 GB of GPU memory and 32 GB of RAM. We implemented early stopping to prevent overfitting. The learning rate was set at 0.001, coupled with a weight decay of 0.001. Dropout rates of 0.3 and 0.5 were



FIGURE 8. Heatmap visualization of GCNN-EC node features across various layers shows clear differentiation. The y-axis categorizes nodes, divided by a blue line between blocks, while the x-axis represents the progression through GCNN-EC layers. Notably, there is minimal feature mixing between blocks as the layers advance, and the connected components become more distinct. This visualization highlights the model's ability to limit over-smoothing, maintaining clearer separations and more defined features across layers, contrary to typical GCN models where increased layer depth blurs feature distinctiveness.

applied across the two layers to enhance generalization. The training utilized cross-entropy loss and the Adam optimizer. Our model underwent training for 100 epochs, with early stopping criteria. The optimal learning rate was determined

through fine-tuning using the Optuna hyperparameter optimization framework [50], ensuring the best possible model performance.

VI. RESULTS

Our GCN-EC model implements dynamic filtering to optimize graph configurations, notably enhancing the formation of distinct connected components. This effect is directly observable in Figure 5, where connected components enriched by node features from their k -hop neighbors are clearly distinguishable.

Further validating our model's performance, we present a comparative analysis of the MedMNIST dataset in Table 2. Our approach achieves performance comparable to that of advanced DNNs, yet it requires significantly fewer parameters, a comparison detailed in Table 4. This demonstrates not only the efficiency of our model but also its ability to maintain high accuracy with a more compact model structure.

Figure 2 demonstrates how features from two connected blocks become indistinguishable as more GCN layers are added, reflecting a typical over-smoothing issue. In contrast, Figure 8 shows that feature mixing is effectively restricted through continuous changes in graph topology, facilitated by the dynamic filter layers. This is further substantiated by Figure 7 highlights how GCNN-EC generates clearly distinguishable graph embeddings, contrasting with the less discriminative embeddings produced by traditional GCNs (Figure 3). This separation visually reinforces our model's ability to mitigate the over-smoothing problem. Furthermore, Figure 6 directly reveals this difference in over-smoothing behavior. We observe that the cosine similarity between the final and pre-final layer features increases more slowly in GCNN-EC compared to standard GCN and GAT models. This slower homogenization of features, coupled with a less drastic decline in accuracy, further underscores the effectiveness of our dynamic filtering approach in creating informative graph-level embeddings.

Our model demonstrates competitive performance against leading CNNs on the MedMNIST dataset, particularly excelling in specific classes, detailed in Table 3 while operating with significantly fewer parameters ranging from 10 to 100 times less as shown in Table 3, it surpasses simple CNN with similar parameter size. Additionally, it surpasses all other GNNs on the MNIST, CIFAR-10, and MSTAR-10 datasets, effectively addressing the critical challenge of over-smoothing and underscoring the robustness of our dynamic filtering approach.

A. DISCUSSION

The proposed GCNN-EC model, incorporating the dynamic filter layer, can be seamlessly integrated with existing GCN layers within the PyTorch Geometric (PyG) package. Other models can also leverage the dynamic filter layer, which can utilize input node features and generate refined features that contribute to a more effective graph representation of images by applying filters over connected nodes and their associated features. In our experiments, we found that an input node feature size of 5 (r, g, b, x, y), subsequently transformed to feature sizes of 16 and 32, yielded optimal

results in terms of creating distinct connected components. The output of the layer can be further processed by additional GCN or other neural network layers to extract higher-level features, demonstrating its versatility and adaptability within existing architectures. This flexibility could be particularly beneficial for medical imaging applications, where there is often a need for low-power and efficient models that can be deployed on edge devices or battery-powered devices (e.g., portable ultrasound machines) [51]. The significantly fewer parameters required by GCNN-EC compared to popular DNNs, as illustrated in Table 4, highlight the potential of GNNs for developing computationally efficient models. This could be particularly advantageous in medical imaging, where computational resources may be limited. Furthermore, the explainability of GNNs, as discussed in [52], is a significant advantage over traditional CNNs, especially in the medical domain where understanding the reasoning behind a model's decisions is crucial for building trust and ensuring safety [53].

VII. CONCLUSION

In this paper, we introduced an innovative dynamic filter layer for GNNs, inspired by spectral graph clustering techniques. This layer significantly improves graph-level classification by focusing on long-range feature aggregation and dynamic edge assignment. Our method is computationally efficient, adaptable to various image types, and demonstrates a clear advantage in creating more discriminative graph representations. Our results convincingly demonstrate that GCNN-EC optimizes graph configurations, leading to the formation of distinct connected components. This not only translates to comparable performance to advanced DNNs on the MedMNIST dataset while requiring significantly fewer parameters, underscoring our model's efficiency and power but also highlights the improved ability of the model to distill information from the images. The fewer parameters and the distinct connected components produced by the model showcase its ability to not only extract information effectively but also maintain the separation of distinct information within the image, contributing to enhanced performance and interpretability.

Crucially, GCNN-EC effectively mitigates the common over-smoothing problem in GCNs. Through dynamic changes in graph topology, our model maintains distinguishable features and generates superior graph-level embeddings. This advantage is further solidified by outperforming leading GNNs across three benchmark datasets.

For future work, we aim to further optimize performance by exploring more sophisticated node representations. Instead of solely relying on individual pixels, we will investigate the use of groups of homogeneous pixels, bounding boxes, and feature-based connections as node candidates. This will involve incorporating feature representations from pre-trained DNNs, or Vision Transformers (ViTs) to leverage both local and global image characteristics. Additionally, we will integrate the dynamic filter layer with more complex

node representations and broaden its compatibility within the PyTorch Geometric framework. This will facilitate extensive testing and validation of the dynamic filter concept's applicability across a wider range of tasks and datasets in the medical imaging domain.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," in *Computer Vision—ECCV 2014* (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2014, pp. 346–361, doi: [10.1007/978-3-319-10578-9_23](https://doi.org/10.1007/978-3-319-10578-9_23).
- [2] S. Gulshad, T. Long, and N. van Noord, "Hierarchical explanations for video action recognition," 2023, *arXiv:2301.00436*.
- [3] P. Rajpurkar, J. Irvin, K. Zhu, B. Yang, H. Mehta, T. Duan, D. Ding, A. Bagul, C. Langlotz, K. Shpanskaya, M. P. Lungren, and A. Y. Ng, "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," 2017, *arXiv:1711.05225*.
- [4] H. Lee and K. Yoon, "Towards better generalization with flexible representation of multi-module graph neural networks," 2022, *arXiv:2209.06589*.
- [5] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," 2013, *arXiv:1312.6203*.
- [6] J. Galon, A. Costes, F. Sanchez-Cabo, A. Kirilovsky, B. Mlecnik, C. Lagorce-Pagès, M. Tosolini, M. Camus, A. Berger, P. Wind, F. Zinzindohoué, P. Bruneval, P.-H. Cugnenc, Z. Trajanoski, W.-H. Fridman, and F. Pagès, "Type, density, and location of immune cells within human colorectal tumors predict clinical outcome," *Science*, vol. 313, no. 5795, pp. 1960–1964, Sep. 2006, doi: [10.1126/science.1129139](https://doi.org/10.1126/science.1129139).
- [7] Y. Shen, B. Zhou, X. Xiong, R. Gao, and Y. Guang Wang, "How GNNs facilitate CNNs in mining geometric information from large-scale medical images," 2022, *arXiv:2206.07599*.
- [8] J. Yang, R. Shi, and B. Ni, "MedMNIST classification decathlon: A lightweight AutoML benchmark for medical image analysis," in *Proc. IEEE 18th Int. Symp. Biomed. Imag. (ISBI)*, Apr. 2021, doi: [10.1109/ISBI48211.2021.9434062](https://doi.org/10.1109/ISBI48211.2021.9434062).
- [9] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Jan. 1998.
- [10] S. K. Zhou, H. Greenspan, C. Davatzikos, J. S. Duncan, B. Van Ginneken, A. Madabhushi, J. L. Prince, D. Rueckert, and R. M. Summers, "A review of deep learning in medical imaging: Imaging traits, technology trends, case studies with progress highlights, and future promises," *Proc. IEEE*, vol. 109, no. 5, pp. 820–838, May 2021, doi: [10.1109/JPROC.2021.3054390](https://doi.org/10.1109/JPROC.2021.3054390).
- [11] M. Z. Alom, C. Yakopcic, T. M. Taha, and V. K. Asari, "Breast cancer classification from histopathological images with inception recurrent residual convolutional neural network," 2018, *arXiv:1811.04241*.
- [12] Q. Liu, M. Kampffmeyer, R. Jenssen, and A.-B. Salberg, "SCG-Net: Self-constructing graph neural networks for semantic segmentation," 2020, *arXiv:2009.01599*.
- [13] A. Singh, P. Van de Ven, C. Eising, and P. Denny, "Connecting the dots: Graph neural network powered ensemble and classification of medical images," 2023, *arXiv:2311.07321*.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015, *arXiv:1512.03385*.
- [15] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," 2016, *arXiv:1608.06993*.
- [16] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," 2019, *arXiv:1905.11946*.
- [17] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaria, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," *J. Big Data*, vol. 8, no. 1, p. 53, Mar. 2021, doi: [10.1186/s40537-021-00444-8](https://doi.org/10.1186/s40537-021-00444-8).
- [18] P. Thanapol, K. Lavanganananda, P. Bouvry, F. Pinel, and F. Leprévost, "Reducing overfitting and improving generalization in training convolutional neural network (CNN) under limited sample sizes in image recognition," in *Proc. 5th Int. Conf. Inf. Technol. (InCIT)*, Oct. 2020, pp. 300–305.
- [19] J. Gu, "Explainability and robustness of deep visual classification models," 2023, *arXiv:2301.01343*.
- [20] P. Elavarthi, J. Lee, and A. Ralescu, "On the ability of CNNs to extract color invariant intensity based features for image classification," 2023, *arXiv:2307.06500*.
- [21] A. Hashemi, Y. Feng, and H. Sabet, "Spherical CNN for medical imaging applications: Importance of equivariance in image reconstruction and denoising," 2023, *arXiv:2307.03298*.
- [22] P. Celard, E. L. Iglesias, J. M. Sorribes-Fdez, R. Romero, A. S. Vieira, and L. Borrajo, "A survey on deep learning applied to medical images: From simple artificial neural networks to generative models," *Neural Comput. Appl.*, vol. 35, no. 3, pp. 2291–2323, Jan. 2023, doi: [10.1007/s00521-022-07953-4](https://doi.org/10.1007/s00521-022-07953-4).
- [23] S. Ira Ktena, S. Parisot, E. Ferrante, M. Rajchl, M. Lee, B. Glocker, and D. Rueckert, "Distance metric learning using graph convolutional networks: Application to functional brain networks," 2017, *arXiv:1703.02161*.
- [24] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, "Geometric deep learning: Going beyond Euclidean data," *IEEE Signal Process. Mag.*, vol. 34, no. 4, pp. 18–42, Jul. 2017, doi: [10.1109/MSP.2017.2693418](https://doi.org/10.1109/MSP.2017.2693418).
- [25] K. Ding, M. Zhou, Z. Wang, Q. Liu, C. W. Arnold, S. Zhang, and D. N. Metaxas, "Graph convolutional networks for multi-modality medical imaging: Methods, architectures, and clinical applications," 2022, *arXiv:2202.08916*.
- [26] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Apr. 2009.
- [27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [28] A. Ortega, P. Frossard, J. Kovačević, J. M. F. Moura, and P. Vandergheynst, "Graph signal processing: Overview, challenges and applications," 2017, *arXiv:1712.00468*.
- [29] *Chebyshev Polynomial Approximates is One Such Approach*.
- [30] R. Vishwakarma, R. B. Joshi, and S. Mishra, "IndoorGNN: A graph neural network based approach for indoor localization using WiFi RSSI," in *Proc. Int. Conf. Big Data Anal.*, in Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2023, pp. 150–165.
- [31] W. Norcliffe-Brown, E. Vafeias, and S. Parisot, "Learning conditioned graph structures for interpretable visual question answering," 2018, *arXiv:1806.07243*.
- [32] B. Huang and K. M. Carley, "Residual or gate? Towards deeper graph neural networks for inductive graph representation learning," 2019, *arXiv:1904.08035*.
- [33] T. Konstantin Rusch, M. M. Bronstein, and S. Mishra, "A survey on oversmoothing in graph neural networks," 2023, *arXiv:2303.10993*.
- [34] B. Y. Weisfeiler and A. A. Leman, "The reduction of a graph to canonical form and the algebra which appears therein," *Tech. Rep.*, 1968.
- [35] Z. Wu, P. Jain, M. A. Wright, A. Mirhoseini, J. E. Gonzalez, and I. Stoica, "Representing long-range context for graph neural networks with global attention," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022.
- [36] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [37] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" 2018, *arXiv:1810.00826*.
- [38] D. Jin, X. Song, Z. Yu, Z. Liu, H. Zhang, Z. Cheng, and J. Han, "BiTe-GCN: A new GCN architecture via Bidirectional Convolution of topology and features on text-rich networks," 2020, *arXiv:2010.12157*.
- [39] U. Alon and E. Yahav, "On the bottleneck of graph neural networks and its practical implications," 2020, *arXiv:2006.05205*.
- [40] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," 2019, *arXiv:1909.03211*.
- [41] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," 2018, *arXiv:1801.07606*.
- [42] M. M. Derakhshani, I. Najdenkoska, T. V. Somsbeek, X. Zhen, D. Mahapatra, M. Worring, and C. G. M. Snoek, "LifeLong: A benchmark for continual disease classification," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, 2022.
- [43] W. Lin, Z. Zhao, X. Zhang, C. Wu, Y. Zhang, Y. Wang, and W. Xie, "PMC-CLIP: Contrastive language-image pre-training using biomedical documents," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, 2023.

- [44] J. Liu, Y. Li, G. Cao, Y. Liu, and W. Cao, "Feature pyramid vision transformer for MedMNIST classification decathlon," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–8.
- [45] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "MedMNIST v2—A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Sci. Data*, vol. 10, no. 1, Jan. 2023.
- [46] Z. Zheng and X. Jia, "Complex mixer for MedMNIST classification decathlon," 2023, *arXiv:2304.10054*.
- [47] F. Di Salvo, S. Doerrich, and C. Ledig, "MedMNIST-C: Comprehensive benchmark and improved classifier robustness by simulating realistic image corruptions," 2024, *arXiv:2406.17536*.
- [48] S. Doerrich, F. Di Salvo, J. Brockmann, and C. Ledig, "Rethinking model prototyping through the MedMNIST+ dataset collection," 2024, *arXiv:2404.15786*.
- [49] O. N. Manzari, H. Ahmadabadi, H. Kashiani, S. B. Shokouhi, and A. Ayatollahi, "MedViT: A robust vision transformer for generalized medical image classification," *Comput. Biol. Med.*, vol. 157, May 2023, Art. no. 106791, doi: [10.1016/j.combiomed.2023.106791](https://doi.org/10.1016/j.combiomed.2023.106791).
- [50] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019.
- [51] P. W. Davros, *Medical Imaging: Principles, Detectors, and Electronics*. Hoboken, NJ, USA: Wiley, Oct. 2009, doi: [10.1118/1.3246610](https://doi.org/10.1118/1.3246610).
- [52] C. Agarwal, O. Queen, H. Lakkaraju, and M. Zitnik, "Evaluating explainability for graph neural networks," *Sci. Data*, vol. 10, no. 1, p. 144, Mar. 2023, doi: [10.1038/s41597-023-01974-x](https://doi.org/10.1038/s41597-023-01974-x).
- [53] A. Chaddad, J. Peng, J. Xu, and A. Bouridane, "Survey of explainable AI techniques in healthcare," *Sensors*, vol. 23, no. 2, p. 634, Jan. 2023, doi: [10.3390/s23020634](https://doi.org/10.3390/s23020634).



ARYAN SINGH received the B.Tech. degree in computer science from Delhi Technological University. He is currently pursuing the Ph.D. degree with the University of Limerick. His professional journey includes brief stints as a Java Developer with MobiKwik and a Backend Developer with Netcore Cloud Solutions, with a focus on data analytics. His Ph.D. research is concentrated on the application of deep learning in medical image analysis and graph-based data representation. He aims to contribute to advancements in data science that benefit medical diagnostics and understanding of image data.



PEPIJN VAN DE VEN received the M.Sc. degree in electronic engineering from Eindhoven University of Technology, The Netherlands, in 2000, and the Ph.D. degree in artificial intelligence for autonomous underwater vehicles from the University of Limerick, in 2005. He is currently a Professor of artificial intelligence with the University of Limerick. His primary research interests include artificial intelligence and machine learning, with a special emphasis on applications in the medical field.



CIARRÁN EISING (Senior Member, IEEE) received the B.E. and Ph.D. degrees from NUI Galway, in 2003 and 2010, respectively. After, he worked in autonomous driving perception with Valeo. He joined the University of Limerick as an Associate Professor, in 2020.



PATRICK DENNY (Member, IEEE) received the B.Sc. degree in experimental physics and mathematics from NUI Maynooth, Ireland, in 1993, and the M.Sc. degree in mathematics and the Ph.D. degree in physics from the University of Galway, Ireland, in 1994 and 2000, respectively. From 1999 to 2001, he was a RF Engineer with AVM GmbH, Germany, where he contributed to developing the RF hardware for the first integrated GSM/ISDN/USB modem. He then worked in supercomputing with Compaq-HP, from 2001 to 2002, before joining Connaught Electronics Ltd. (later Valeo), Galway, Ireland, as the Team Leader of RF design. In 2010, he was appointed as an Adjunct Professor of engineering and informatics with the University of Galway. In 2022, he became a Lecturer in artificial intelligence with the Department of Electronic and Computer Engineering, University of Limerick, and in 2024, he advanced to an Associate Professor of artificial intelligence and imaging with the Department of Computer Science and Information Systems (CSIS), University of Limerick. His Ph.D. conducting a part of research with GFZ-Potsdam, Germany. Over the following two decades, he led efforts in developing innovative RF and imaging systems, including the first mass-production HDR automotive cameras for prominent car manufacturers, such as Jaguar Land Rover, BMW, and Daimler. He is also the Co-Founder and a Committee Member of the IEEE P2020 Automotive Imaging Standards Group, the AutoSens Conference on Automotive Imaging, and the IS&T Electronic Imaging Autonomous Vehicles and Machines (AVM) Conference.

...